Instituto Tecnológico de Costa Rica

Escuela Ingeniería en Computadores

Curso: CE1104 Fundamentos de sistemas computacionales

Documentación técnica Stranger-Tec

Estudiantes:

Gabriel Picado

Gabriel Ramírez

Profesor: Leonardo Araya Martínez

Semestre I 2026

0. Índice

Contenido

0. Indice ..... Error! Bookmark not defined.
1. Introducción **3**
1.1. Propósito del documento 3
1.2. Descripción del proyecto 3
1.3 Objetivos y Alcance 3
1.4 Definiciones, Acrónimos y Abreviaturas 3
2. Descripción General del Sistema **5**
2.1. Perspectiva del Producto 5
2.2. Funcionalidades principales 5
2.3. Restricciones del sistema, Supuestos y Dependencias 5
3. Requisitos del Sistema **7**
3.1. Requisitos Funcionales 7
3.2. Requisitos No Funcionales 8
4. Diseño de la Solución **9**
4.1. Visión General de la Arquitectura 9
4.2. Descripción de Componentes 9
4.4. Diagrama de Clases y Modelo de Datos ..... **Error! Bookmark not defined.**
4.5. Diagrama de Flujo y Secuencia 12
4.6. Decisiones de Diseño y Justificación 13
5. Implementación y Validación de Calidad **14**
5.1. Estructura del repositorio 14
5.2. Entorno de Desarrollo, Guía de instalación y Ejecución 14
5.3. Estrategias y Casos de Prueba 15
5.4. Resultados de pruebas y problemas encontrados 15
6.1. Herramientas de IA utilizadas 18
6.3. Declaración de Responsabilidad y Autoría 19
7. Conclusión **20**
8. Bibliográfia **21**

1. Introducción

1.1. Propósito del documento

Este documento describe el diseño, estructura y la implementación del proyecto StrangerTEC, que es un juego interactivo de comunicación en código Morse. La idea del documento es explicar brevemente como se desarrolló y entender el funcionamiento del proyecto.

1.2. Descripción del proyecto

StrangerTEC es un código creado como un juego interactivo para ser jugado entre 2 jugadores, cuenta con una “maqueta” inspirada en la pared de luces de la serie Stranger Things, la cual es controlada por una Raspberry Pi Pico W, y una interfaz gráfica desarrollada en Python. El sistema permite que dos jugadores interpreten una frase y la repliquen en código Morse, utilizando leds y un buzzer.

La modalidad permite que ambos jugadores pasen un código en la maqueta con la Raspberry y otro código con la interfaz creada en Python. Para lo cual se creó la lógica del juego de tal manera que sea una competencia entre estos dos jugadores.

1.3 Objetivos y Alcance

Objetivo principal: Desarrollar un sistema interactivo que permita a dos jugadores comunicarse y competir utilizando el código Morse, integrando hardware (Raspberry, leds, buzzer, botones) con software (Python, Tkinter, Thonny).

Objetivos específicos:

- Implementar luces LED en un panel funcional para transmitir un mensaje.
- Desarrollar la lógica de codificación de código Morse en Python.
- Construir una interfaz gráfica en la computadora para la representación del juego.
- Establecer una conexión tanto por Wi-Fi como por USB entre la Raspberry Pi Pico W y la computadora.
- Implementar un sistema de puntaje y retroalimentación al jugador.

Alcance: El sistema cubre el modo local y el modo de trasmisión simple. El modo Versus (entre 2 maquetas) y el modo escucha y transmisión quedaron fuera del proyecto trabajado.

1.4 Definiciones, Acrónimos y Abreviaturas

Término	Definición
PWM	Modulación por Ancho de Pulso.
Raspberry Pi Pico W	Microcontrolador principal del sistema físico.
Tkinter	Biblioteca estándar de Python para interfaces graficas

2. Descripción General del Sistema

2.1. Perspectiva del Producto

StrangerTec es un sistema que integra hardware y software. La maqueta física funciona como servidor de eventos (captura entradas del jugador con botones y ejecuta salidas con LEDs y buzzer), mientras que la aplicación de la computadora actúa como control (envia frases, recibe respuestas y calcula puntajes), aunque tambien puede enviar señales con el teclado. La comunicación entre ambos sistemas se realiza a través del USB y mediante Wi-Fi.

El sistema no depende de servicios externos ni datos en la nube. Toda la lógica y la lista de frases está dentro del código de la PC, y el estado de la partida se gestiona en memoria al mismo tiempo conforme sucede.

2.2. Funcionalidades principales

- Selección aleatoria de frases de una lista con un máximo de 10 frases (máximo 16 caracteres cada una).
- Visualización de la frase mediante señales luminosas.
- Captura de entrada Morse del jugador mediante el botón pulsador.
- Comparación entre la entrada del jugador y la frase original con puntaje por carácter correcto.
- Interfaz grafica de escritorio con visualización de puntaje y pantalla final de resultados.
- Cambio de turno entre los dos jugadores al finalizar cada intento.

2.3. Restricciones del sistema, Supuestos y Dependencias

Restricciones:

- El sistema empotrado debe ser una Raspberry Pi Pico W programada en MicroPython.
- La interfaz grafica de la computadora debe estar programada en Python utilizando bibliotecas estándar permitidas (Tkinter, Pygame, etc.).
- Las frases tienen un máximo de 16 caracteres y solo pueden contener los caracteres definidos en el diccionario Morse del proyecto
- No fue implementado el modo escucha y transmisión ni el modo versus entre dos maquetas.
- El proyecto se desarrolló en parejas.

Supuestos:

- El equipo con la aplicación tiene mínimo Python 3.10 instalado.
- La Raspberry Pi Pico W se concecta ya sea mediante ip o cable USB.
- El usuario conoce los fundamentos básicos de código Morse o los puede tener a mano a la hora de jugar.

Dependencias:

- Python 3.10+: Tkinter
- MicroPython (Pico W): machine, machine.Pin, machine.PWM, time, network, socket, _thread
- Hardware: Raspberry Pi Pico W, LEDs, buzzer, boton pulsador. Registro de corrimientos

3. Requisitos del Sistema

3.1. Requisitos Funcionales

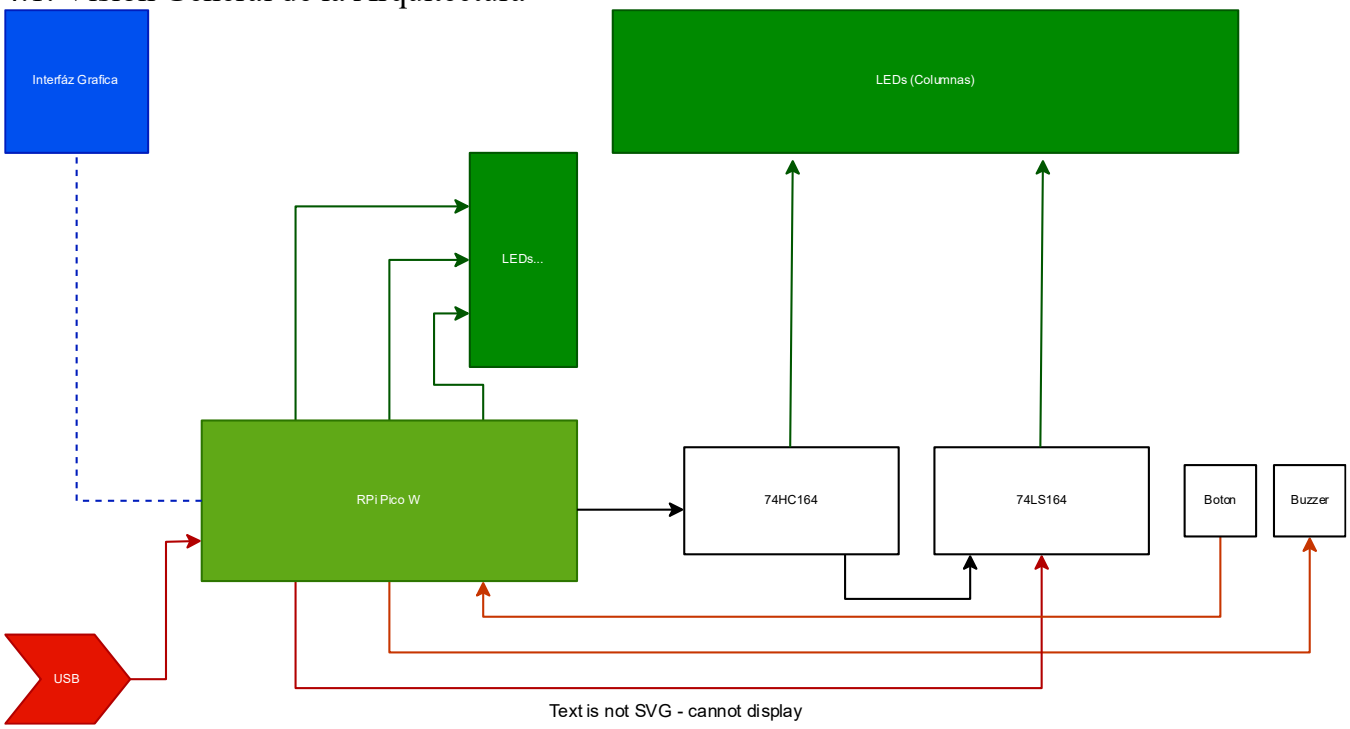
ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	El sistema debe seleccionar aleatoriamente una frase de la lista y enviarla a la maqueta.	Alta	La frase enviada coincide con la mostrada en la maqueta (LEDs/buzzer).
RF-002	El panel de LEDs debe encenderse en el orden correcto para representar cada carácter Morse.	Alta	Cada letra/número se muestra siguiendo la convención Morse del proyecto.
RF-003	El buzzer debe reproducir los puntos y rayas de cada carácter con la temporización correcta.	Alta	Los tiempos de punto, raya, pausa entre símbolos, caracteres y palabras son los especificados.
RF-004	El botón pulsador debe detectar punto (presión corta) y raya (presión larga).	Alta	Presión < umbral = punto; presión >= umbral = raya.
RF-005	El sistema debe detectar las pausas entre símbolos y entre letras.	Alta	Pausa de 1 unidad separa símbolos; pausa de 3 unidades separa letras.
RF-006	El buzzer debe sonar durante toda la presión del botón como retroalimentación auditiva.	Media	El buzzer emite sonido desde que se presiona hasta que se suelta el botón.
RF-007	El sistema debe comparar la entrada Morse del jugador con la frase original y asignar puntaje.	Alta	El puntaje refleja correctamente el número de caracteres acertados.
RF-009	La interfaz gráfica debe mostrar el puntaje de cada jugador en tiempo real.	Media	El marcador se actualiza tras cada carácter evaluado.
RF-010	El sistema debe mostrar una pantalla final con los puntajes totales e indicar el ganador.	Alta	Al finalizar ambos turnos, se despliega el resultado final con el ganador.

3.2. Requisitos No Funcionales

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Rendimiento	El sistema debe responder a la presión del botón sin retardo perceptible.	Latencia de detección < 50 ms.
RNF-002	Usabilidad	La interfaz debe ser comprensible para usuarios sin experiencia previa en Morse.	El jugador puede completar una ronda sin instrucciones adicionales tras una explicación inicial.
RNF-003	Confiabilidad	El sistema debe mantener el estado del juego sin pérdidas de datos durante una partida.	No se pierden puntos ni se reinicia el juego de forma inesperada.
RNF-004	Mantenibilidad	El código debe estar comentado para que cualquier integrante pueda orientarse.	Cada función o bloque lógico tiene comentarios descriptivos.
RNF-005	Portabilidad	La aplicación de PC debe ejecutarse en Windows 10+ con Python 3.10+.	El sistema se ejecuta correctamente sin modificaciones adicionales.

4. Diseño de la Solución

4.1. Visión General de la Arquitectura



Este diagrama muestra de manera general, como se acomodó la maqueta física. Se siguió el diagrama dado por el profesor.

4.2. Descripción de Componentes

Componente	Responsabilidad	Tecnología	Dependencias
interfazgrafica.py (branch: PRUEBA-JUEGO)	Módulo principal de la interfaz PC. Integra: pantalla de inicio con conexión WiFi a la Pico W (via HTTP), ingreso de nombres de jugadores (tk.Toplevel), selección de velocidad (Unidades A=0.2s, B=0.3s, C=0.5s), lógica de juego por turnos, captura de Morse por teclado (SPACE), decodificación, cálculo de	Python / Tkinter, requests	Pico W (HTTP GET sobre WiFi)

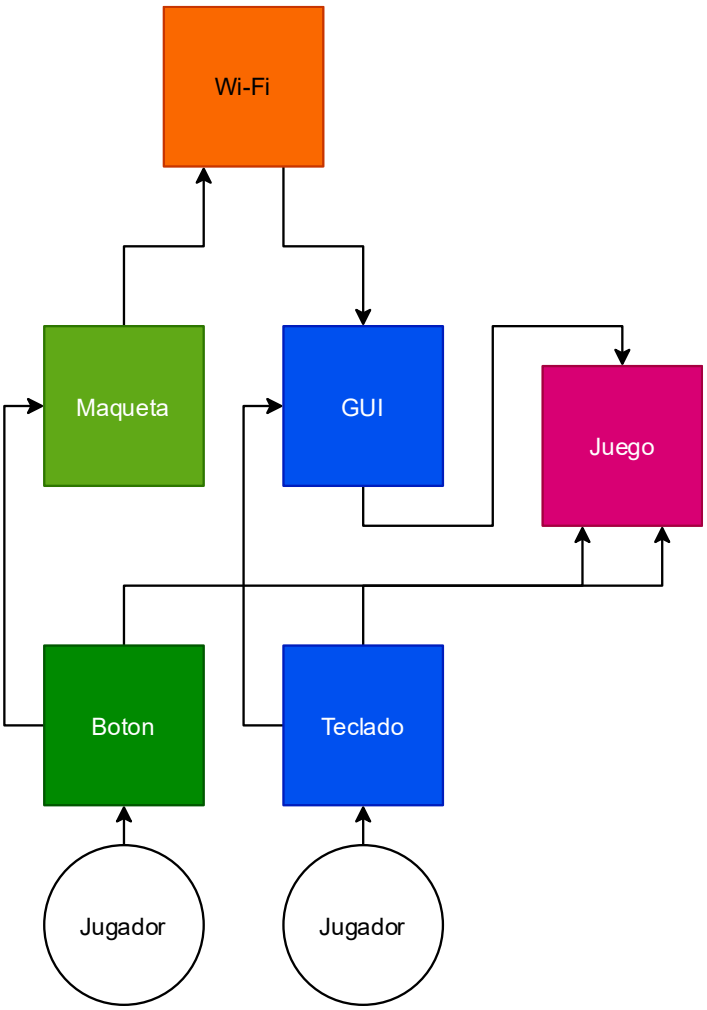
	puntaje por carácter correcto y polling de respuesta de la Pico W cada 2 segundos.		
config.py (branch: interfaz-grafica)	Archivo de configuración de la interfaz gráfica: constantes globales, lista de frases (FRASES), parámetros de conexión y ajustes de velocidad reutilizables por los demás módulos de la PC.	Python	interfazgrafica.py
index.html (branch: interfaz-grafica)	Página web de respaldo para visualización del estado del juego o interfaz alternativa servida desde la Pico W.	HTML	Servidor HTTP en Pico W
main.py (branch: PRUEBA-JUEGO / Pico W)	Firmware completo de la Raspberry Pi Pico W. Inicializa hardware: GPIO 15 (botón, PULL_DOWN), GPIO 17 (buzzer PWM 500 Hz), GPIO 18/19 (data/clock del 74HC164), GPIO 12/13/14 (filas LED). Lanza dos hilos paralelos con _thread: servidor HTTP en puerto 80 (atiende GET_MORSE, SET_UNIDAD, LED_ON/OFF, SYS_OFF) y control físico del botón Morse. Detecta punto (duracion <= 1u) y raya (duracion >= 2.5u), emite tonos diferenciados (800 Hz punto / 400 Hz raya), gestiona separadores de letra (3u) y palabra (7u), e ilumina el LED correspondiente al carácter decodificado.	MicroPython	network, socket, _thread, machine, utime
shift_register / iluminar_letra() (en main.py)	Función shift_out_16() que envía 16 bits en serie a los dos 74HC164 en daisy-chain (MSB primero) usando GPIO 18 (data) y GPIO 19 (clock). La función iluminar_letra() consulta el mapa CHAR_TO_BIT (39	MicroPython	machine.Pin

	caracteres) y activa la columna correcta del shift register más la fila correspondiente (GPIO 14 = fila A/C/E..., GPIO 13 = fila B/D/F..., GPIO 12 = números/signos).		
Tablas MORSE (en main.py)	Diccionario LETRA_A_MORSE con 39 entradas (A-Z, 0-9, +, -, espacio→'/') y su inverso MORSE_A_LETRA para decodificación. Seguido del mapa CHAR_TO_BIT que asocia cada carácter a su columna de LED (0-12) y los sets ODD_LETTERS, EVEN_LETTERS y NUMBERS para selección de fila.	MicroPython	-

4.3. Stack Tecnológico

Categoría	Tecnología	Versión	Propósito
Lenguaje (PC)	Python	3.10+	Lógica del juego e interfaz grafica
Framework GUI	Tkinker	Estandar	Interfaz gráfica de escritorio
Lenguaje (empotrado)	MicroPython	1.22+	Firmware de la Raspberry Pi Pico W
Hardware principal	Raspbrerry Pi Pico W	RP2040-W	Microcontrolador del sistema empotrado
IDE empotrado	Thonny	4.1.7	Desarrollo y carga de firmware MicroPython

4.4. Diagrama de Flujo y Secuencia



La secuencia del proyecto es la siguiente:

1. Se conecta la maqueta con Wi-Fi
2. Se conecta la maqueta con la GUI
3. Se Inicia el juego
4. A los jugadores se les enseña la palabra
5. Los jugadores deberán escribir la palabra en código morse correctamente.
6. El morse escrito llega a la plataforma respectiva.
7. Se termina la ronda y se calculan los puntos de cada jugador.

4.5. Decisiones de Diseño y Justificación

Decisión	Alternativas Evaluadas	Opción Elegida	Justificación
Lenguaje de interfaz PC	PyQt, wxPython, Tkinter	Tkinter	Biblioteca estándar de Python, sin dependencias externas adicionales y suficiente para los requerimientos del proyecto.
Control de LEDs	Control directo pin-a-pin, Registro de corrimiento	74HC164 (registro de corrimiento)	Permite controlar 8 LEDs con solo 2 pines del microcontrolador, liberando pines para otros componentes.
Comunicación PC-Maqueta	WiFi, Bluetooth, USB serial	USB serial, WiFi	Menor latencia, sin configuración de red, más confiable para el Modo Local.
Temporización Morse	Hilos (threading), máquina de estados, polling, uasyncio	_thread	Permite correr simultáneamente el código necesario para la red, y lo físico.
Almacenamiento de frases	Archivo externo, base de datos, lista en código	Lista en código Python (PC)	Suficiente para 10 frases. Modificable fácilmente entre partidas sin infraestructura adicional.

5. Implementación y Validación de Calidad

5.1. Estructura del repositorio

```
/proyecto-raiz
          ├── src/ -> Codigo final
                ├── rpi-main.py -> Codigo para la Raspberry Pi Pico W
                ├── StrangerTEC-GUI.py -> Codigo de la interfaz grafica
                └── Documento Tecnico.pdf -> El documento actual.
```

5.2. Entorno de Desarrollo, Guía de instalación y Ejecución

Lista los prerrequisitos de software necesarios para ejecutar el proyecto en un entorno local de desarrollo. Además, se proporcionan instrucciones paso a paso para que cualquier persona pueda ejecutar el proyecto desde cero. Incluye comandos exactos.
Ejemplo:

- Entorno:
  - Sistema Operativo: [Windows 10+, macOS 12+, Ubuntu 20.04+]
  - Lenguaje / Runtime: [Python 3.10+, MicroPython (Raspberry Pi Pico)]

- Guía de instalación y ejecución:
  - Clonar repositorio de GitHub:
    `git clone https://github.com/unmolten/StrangerTEC-GP-GR`
  - Instalar dependencias:
    `pip install requests`
  - Correr código:
    Correr `rpi-main.py` en Thonny conectado a la Raspberry Pi Pico W
    Correr `StrangerTEC.py` en un IDE (Preferiblemente Visual Studio Code.)

5.3. Estrategias y Casos de Prueba

Describe la estrategia general de pruebas adoptada: tipos de prueba, herramientas utilizadas, cobertura, objetivo y responsables. Adicionalmente, se definen y documentan los casos de prueba más representativos con identificador, precondiciones, pasos, resultado esperado y resultado obtenido.
Ejemplo:

- Estrategias de pruebas:

Tipo de Prueba	Descripción	Herramienta	Cobertura Objetivo
Pruebas Unitarias	Verificación de funciones y métodos individuales de forma aislada.	Creación de branches con parte del código para implementar lentamente las funciones	Código de la interfaz grafica y Código de la Raspberry Pi Pico W
Pruebas Funcionales	Validación del comportamiento del sistema desde la perspectiva del usuario final.	Pruebas Físicas con el código y la maqueta	Casos de uso principales
Pruebas de Rendimiento	Medición del comportamiento bajo diferentes circunstancias.	Pruebas Físicas con el código y la maqueta	Interacción con los jugadores para una experiencia agradable

- Casos de prueba:

ID	Descripción	Pasos	Resultado Esperado	Resultado Obtenido	Estado
CP-001	Conexión con internet fallida	1. Se corre el código de la Raspberry Pi con credenciales incorrectos	Imprime en la consola un error	Sigue intentando conectar con credenciales dados	☒ No Pasó
CP-002	Iniciar Juego sin nombres	1. No llenar campos para nombres de jugadores.	El sistema muestra un mensaje de error.	El sistema no continuó sin nombres para los jugadores	✅ Pasó
CP-003	Código Morse No valido	1. Empezar el juego, 2. Ingresar código morse sin valor alguno o incorrecto	Crear un “?” para indicar que no se detectó código morse adecuado	Remplaza el código morse invalido con un “?”	✅ Pasó

5.4. Resultados de pruebas y problemas encontrados

Ejemplo:

- Resultados:
    - Casos de prueba exitosos:
        - Mayoría de las pruebas unitarias funcionaron
        - Se puede jugar fluidamente
    - Imágenes del programa corriendo:

      Imagen 1: La maqueta finalizada con el sistema de LEDs Funcionando (Note que se tuvo que remplazar un 74HC164 por un 74LS164 Debido a que estaba defectuoso, por ende el espacio vacio.)

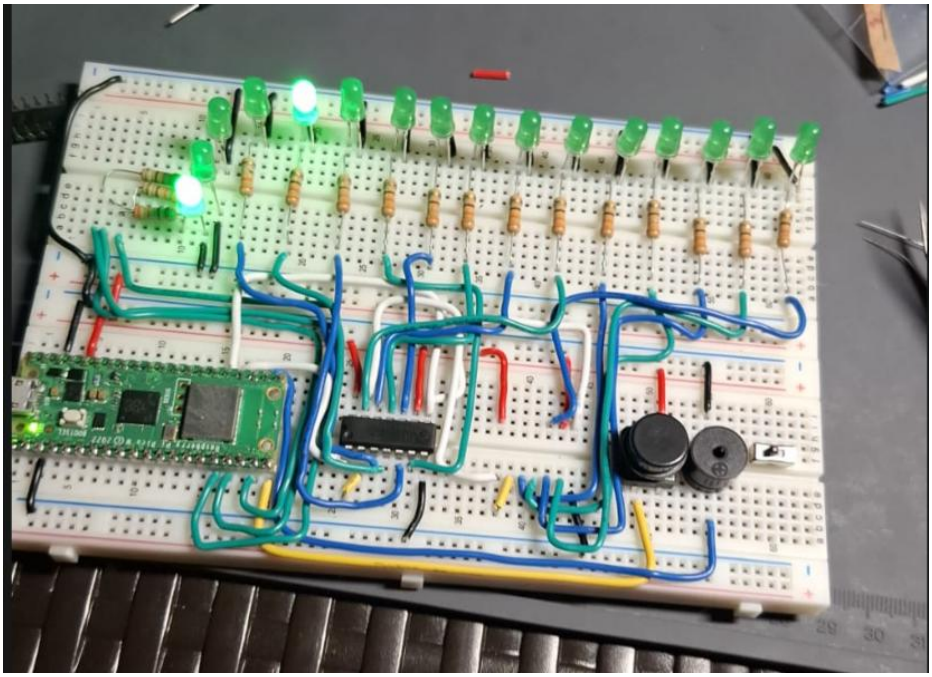
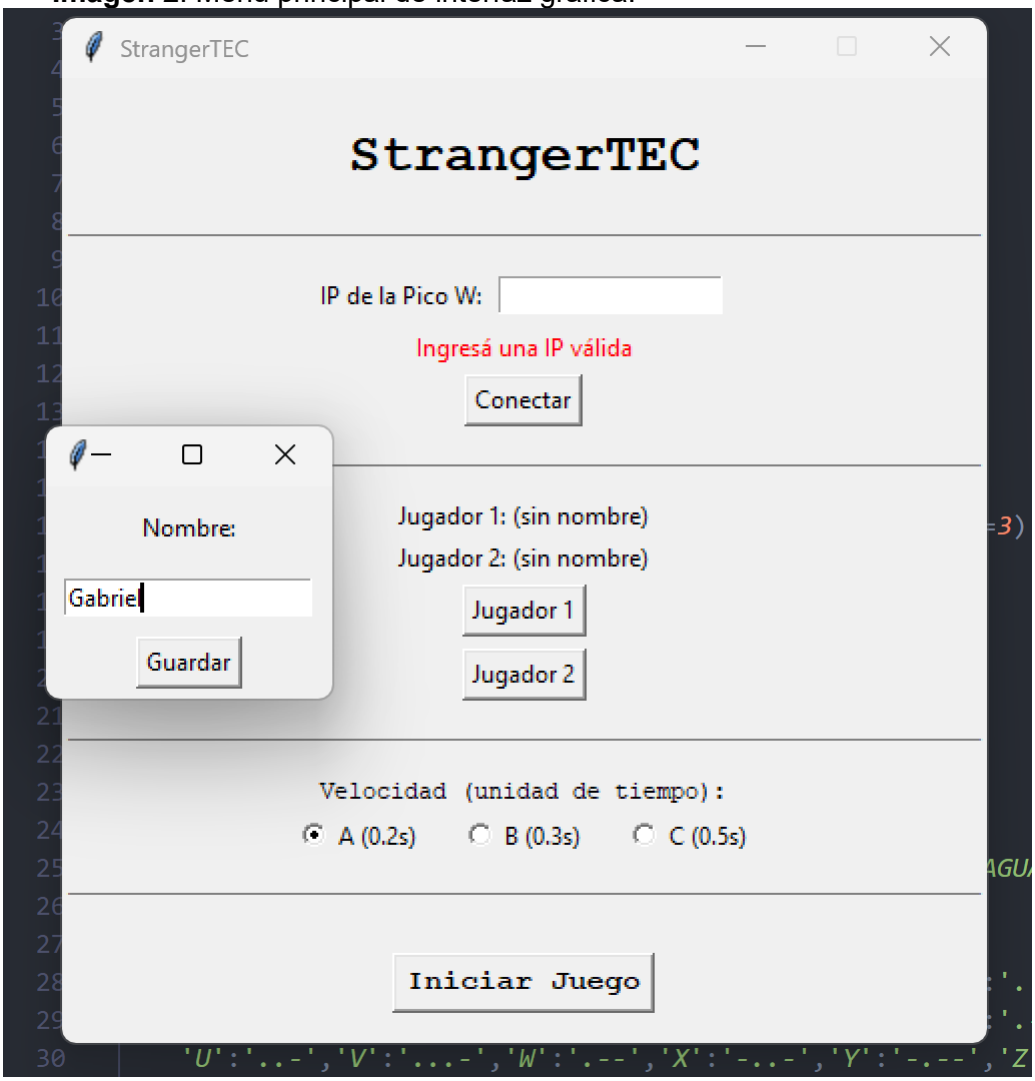


Imagen 2: Menú principal de interfaz grafica.



Problemas encontrados:

- Problemas Solucionados:
    - Problema de funcionalidad de botones y red: Inicialmente al conectar un boton y un led por medio de la Raspberry Pico Pi W, a la vez de crear una conexión a internet, fallaba el boton. Se soluciono usando la librería "_thread" y corriendo simultáneamente el código físico y de Wi-Fi
    - Problema del segundo registro de corrimiento: Al probar la maqueta final, se encontró que uno de los registros de corrimiento no funcionaba. Se soluciono remplazándolo por un registro de corrimientos 74LS164.

6. Declaración de uso de Inteligencia Artificial y herramientas

6.1. Herramientas de IA utilizadas

Herramienta de IA	Tipo	Propósito
Claude (Sonnet 4.6)	Modelo de lenguaje (LLM)	Ayuda con el código en partes complicadas.
Gemini (Gemini 3)	Modelo de lenguaje (LLM)	Ideas generales de como continuar.
ChatGPT (GPT-5.3)	Modelo de lenguaje (LLM)	Revisión básica de la documentación

6.2. Descripción del Uso de Inteligencia Artificial

- Claude:
  Generación y asistencia en código en partes difíciles, o con pocos recursos disponibles. Ej. Funcionalidad 74HC164 o Ayuda con _thread.
- ChatGPT:
  Revisión simple de los documentos para adecuada ortografía y estructura.
- Gemini:
  Ayuda con ideas básicas para partes no intuitivas del proyecto. Ej. Opción de usar la librería “requests”

6.3. Declaración de Responsabilidad y Autoría

DECLARACIÓN DE AUTORÍA Y RESPONSABILIDAD

Los suscritos, integrantes del equipo de trabajo del proyecto "StrangerTEC", declaramos que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.
2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido por los integrantes del equipo.
4. Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

7. Conclusión

El proyecto StrangerTEC logró integrar exitosamente hardware embebido y software de escritorio en un sistema funcional de comunicación Morse. Se cumplieron los objetivos principales: el panel de LEDs responde correctamente a las entradas del jugador, la decodificación Morse en tiempo real, y la interfaz gráfica gestiona el flujo del juego entre ambos jugadores de forma estable.

El desarrollo implicó superar obstáculos técnicos significativos, principalmente en la etapa de hardware. El diagnóstico y reemplazo de un registro de corrimiento defectuoso, la correcta alimentación del 74LS164 mediante VBUS, y la resolución de problemas de configuración, fueron aprendizajes concretos que reforzaron la comprensión del funcionamiento de circuitos digitales reales más allá de la teoría.

A nivel de software, el uso de _thread para ejecutar en paralelo el servidor Wi-Fi y la lectura física del botón fue una decisión clave que resolvió uno de los problemas más tempranos del proyecto. La arquitectura cliente-servidor sobre HTTP resultó suficiente y práctica para la comunicación entre la Pico W y la interfaz gráfica, sin necesidad de dependencias externas complejas.

Como área de mejora, el proyecto se beneficiaría de una mejor planificación del tiempo de pruebas físicas, ya que varios problemas de hardware surgieron tarde en el desarrollo. A futuro, implementar los modos pendientes como el modo Versus entre dos maquetas representaría una expansión natural y viable sobre la base ya construida.

En conclusión, el proyecto StrangerTEC cumplió con los objetivos planteados, logrando combinar, la parte física y digital en una experiencia de juego interactiva. A lo largo del desarrollo se superaron obstáculos técnicos relevantes, desde problemas de concurrencia en el firmware hasta fallas físicas en componentes de hardware, cada uno de los cuales representó un aprendizaje concreto sobre el diseño y la implementación de sistemas computacionales reales.

8. Bibliográfia

Fuentes usadas para investigación:

Funcionalidad PULL_DOWN:
https://www.youtube.com/watch?v=5vnW4U5Vj0k

Funcionalidad 74HC164:
https://www.youtube.com/watch?v=8PvnC65E2XI
https://www.instructables.com/The-74HC164-Shift-Register-and-your-Arduino/

Datos sobre 74HC164:
https://www.ti.com/lit/ds/symlink/sn74hc164.pdf

Uso de buzzer pasivo con MicroPython:
https://www.youtube.com/watch?v=tqXARSYouLE

Diagrama de pines GPIO Raspberry Pico Pi W:
https://www.alexdwilson.dev/learning-in-public/gpio-on-the-raspberry-pi-pico

Conexión a internet Raspberry Pico Pi W:
https://projects.raspberrypi.org/en/projects/get-started-pico-w/0
https://www.youtube.com/watch?v=wFi9NeP-McE
https://www.youtube.com/watch?v=2KDSbA3dMiY
https://www.youtube.com/watch?v=fqsOZ_FVgfg

Documentación Librería “socket”:
https://docs.python.org/3/library/socket.html

Documentación Librería “_thread”:
https://docs.python.org/3.13/library/_thread.html#module-_thread

Idea para el sistema de código morse en Python:
https://www.geeksforgeeks.org/python/morse-code-translator-python/

Concatenación de Strings en Python:
https://www.w3schools.com/python/python_strings_concatenate.asp

Diccionarios en Python:
https://www.w3schools.com/python/python_dictionaries.asp